Student Name: Maksuda Yasmin

Student ID: 2026-1-96-026

Department of Computer Science And Engineering

University Name :East West University

Project 6: Bangla Clarification Questions for Database Queries./

Bangla Clarify SQL: Generating Clarification Questions for Ambiguous Bangla and Banglish Database Queries

---

Abstract:

Natural-language interfaces to databases allow users to retrieve information without writing SQL queries. However, natural-language questions are frequently ambiguous, particularly when users omit important information such as time period, entity identity, aggregation operation, comparison threshold, or sorting direction. This problem becomes more challenging for Bangla and Banglish queries because of linguistic variation, code-mixing, and informal expressions. This project proposes **BanglaClarifySQL**, a clarification-aware Text-to-SQL framework that identifies ambiguity in Bangla and Banglish database queries and generates concise clarification questions before producing the final SQL query. A self-constructed dataset of approximately 1,500–2,000 ambiguous queries will be developed from existing Text-to-SQL resources such as Spider and BIRD. The study compares a rule-based clarification system, a direct Text-to-SQL large language model, and an LLM-based clarification system, with a few-shot schema-aware system as an optional baseline. Performance will be evaluated using ambiguity classification F1, clarification relevance and completeness, human preference, SQL exact match, execution accuracy, latency, and token cost. Controlled ablation studies will investigate the effects of schema information, prompting strategy, explicit ambiguity labels, and language form. The study aims to determine whether clarification improves SQL execution accuracy sufficiently to justify the additional computational cost.

Keywords— Bangla NLP, Banglish, Text-to-SQL, clarification questions, large language models, database question answering, ambiguity detection

I. INTRODUCTION AND MOTIVATION

Natural-language interfaces for relational databases have become an important research area because they allow users to query structured data without requiring knowledge of SQL. Text-to-SQL systems translate natural-language questions into executable SQL queries. However, a major limitation of such systems is that natural-language questions may be incomplete or ambiguous.

For example, the Bangla query *"সবচেয়ে বেশি বিক্রি হওয়া পণ্যটি দেখাও"* ("Show the product with the highest sales") does not specify the time period. A Text-to-SQL model may incorrectly assume a particular year or attempt to generate SQL without the information required to produce a unique answer. Similar ambiguity can occur with entity identity, aggregation, comparison thresholds, sorting direction, and vague adjectives such as *ভালো* ("good"), *বেশি* ("high"), *কম* ("low"), and *সেরা* ("best"). These ambiguity types are identified as central challenges in the proposed project.

The problem is potentially more difficult for Bangla and Banglish users. Banglish commonly mixes Bangla words with English words and may contain substantial spelling variation. Consequently, a system that directly generates SQL may fail even when the underlying database schema is sufficient to answer the user's intended question.

This project proposes a clarification-aware approach. Instead of immediately generating SQL, the system first determines whether the query is ambiguous, identifies the missing information, and asks an appropriate clarification question. After obtaining the user's response, the system generates the final SQL query.

The main research question is:

How reliably can multilingual language models identify ambiguity in Bangla and Banglish database queries and generate relevant clarification questions that improve final SQL execution accuracy?

The project investigates four additional questions: whether database schema information improves clarification quality, whether clarification improves SQL execution accuracy compared with direct Text-to-SQL, which prompting strategy performs best, and whether the improvement in accuracy justifies the additional latency and token cost.

II. RELATED WORK

Text-to-SQL research has produced several important datasets and modeling approaches. **Spider** introduced a large cross-domain benchmark for translating natural-language questions into SQL over multiple relational databases [1]. Its cross-domain design makes it suitable as a source of database schemas and answerable questions for constructing the proposed dataset.

RAT-SQL introduced relation-aware schema encoding for Text-to-SQL and demonstrated the importance of modeling relationships between tables, columns, and question tokens [2]. This motivates the use of explicit database schema information in the proposed clarification system.

PICARD proposed constrained decoding for language models to improve the validity of generated SQL queries [3]. While PICARD focuses on controlling SQL generation, BanglaClarifySQL focuses on an earlier problem: determining whether sufficient information exists to generate the correct SQL in the first place.

DIN-SQL investigated decomposition and prompting strategies for Text-to-SQL using large language models [4]. Its decomposition-oriented approach is relevant to the proposed multi-stage process, where ambiguity detection and clarification precede final SQL generation.

More recently, **BIRD** introduced a challenging Text-to-SQL benchmark involving large and realistic databases [5]. BIRD can provide additional database schemas and questions for constructing and evaluating the proposed dataset.

Although these resources provide strong foundations for Text-to-SQL research, the proposed project focuses specifically on **ambiguity detection and clarification-question generation for Bangla, Banglish, and mixed-language database queries**. The project therefore extends the conventional direct Text-to-SQL setting by introducing a clarification stage.

III. RESEARCH QUESTION AND PROBLEM FORMULATION

A. Research Question

The primary research question is:

How reliably can multilingual language models identify ambiguity in Bangla and Banglish database queries and generate relevant clarification questions that improve final SQL execution accuracy?

The study evaluates whether a clarification-aware system can outperform direct Text-to-SQL when the user's query does not contain enough information to determine a unique SQL interpretation.

B. Ambiguity Types

The project considers six major ambiguity categories:

1. **Time Period:** Missing year, month, date, or time range.
2. **Entity Identity:** Unclear identification of an entity with multiple possible matches.

3. **Aggregation Operation:** Unclear whether the user wants sum, average, maximum, minimum, or another aggregation.
4. **Comparison Threshold:** Unclear meaning of expressions such as “high price” or “more sales.”
5. **Sort Order:** Unclear ascending or descending ordering.
6. **Vague Adjective:** Ambiguous expressions such as “good customer,” “best product,” or “low salary.”

For example, *“ভালো customer-দের দেখাও”* (“Show the good customers”) does not specify what makes a customer “good.” A useful clarification could ask whether “good” means customers with the highest purchase amount or the highest number of orders.

IV. DATASET AND PREPROCESSING

A. Dataset Construction

A self-constructed dataset will be developed because the project requires data specifically focused on ambiguity and clarification questions in Bangla and Banglish Text-to-SQL. The initial target size is approximately **1,500–2,000 queries**.

Answerable Text-to-SQL questions will first be selected from publicly available datasets such as Spider and BIRD. These questions will then be translated or adapted into:

- Bangla;
- Banglish; and
- mixed Bangla-English expressions.

Controlled ambiguity will subsequently be introduced by removing one or more pieces of information necessary for determining the intended SQL query.

A representative example is:

Original:
“২০২৪ সালের সবচেয়ে বেশি বিক্রি হওয়া পণ্যটি দেখাও।”

Ambiguous:
“সবচেয়ে বেশি বিক্রি হওয়া পণ্যটি দেখাও।”

Missing information: Time period.

Gold clarification:
“আপনি কোন সময়ের বিক্রির তথ্য জানতে চান—২০২৪ সাল, নাকি অন্য কোনো সময়?”

B. Dataset Distribution

The proposed dataset will contain approximately:

Language	Target Queries
Bangla	500
Banglish	500
Mixed Bangla-English	500
Additional/generalization examples	0–500
Total	**1,500–2,000**

C. Annotation

Each instance will contain the query ID, language, database/schema ID, original SQL, ambiguous query, ambiguity type, missing information, gold clarification question, possible user response, correct final SQL, and difficulty level.

At least two human annotators will independently annotate the data. Disagreements will be resolved through discussion or a third annotator. Cohen's Kappa may be used to measure inter-annotator agreement.

D. Preprocessing

The preprocessing pipeline will include:

1. Normalizing Bangla text and Unicode representations.
2. Preserving meaningful Banglish and code-mixed expressions.
3. Associating each query with its database schema.
4. Checking that the original SQL is executable.
5. Introducing controlled ambiguity.
6. Verifying the ambiguity label and missing information through human annotation.
7. Creating training, validation, and test splits.

The proposed split is **70% training, 15% validation, and 15% testing**, with the test set containing unseen queries, ambiguity types, languages, and database schemas.

V. METHODOLOGY

The proposed system follows a multi-stage clarification-aware Text-to-SQL architecture.

A. Baseline 1: Rule-Based Clarification

The first baseline uses manually designed rules to identify common ambiguity patterns. For example, absence of a time expression may trigger a question about the time period, while expressions such as “best,” “high,” or “low” may trigger a question about ranking or thresholds.

This baseline is simple, interpretable, and reproducible but is expected to have difficulty with complex linguistic ambiguity.

B. Baseline 2: Direct Text-to-SQL LLM

The second baseline receives the natural-language query and database schema and directly generates SQL without asking clarification questions.

This provides the primary comparison for determining whether the proposed clarification stage actually improves SQL performance.

C. Proposed LLM-Based Clarification System

The proposed system receives:

- ambiguous Bangla/Banglish query;
- database schema; and
- task-specific prompt.

The LLM first determines whether the query is ambiguous, identifies the ambiguity type and missing information, and generates one concise clarification question. A simulated user response containing the gold missing information is then supplied to the system, after which the model generates the final SQL query.

D. Optional Few-Shot Schema-Aware Baseline

An additional baseline may provide examples containing an ambiguous query, ambiguity type, clarification question, user response, and final SQL. This tests whether few-shot demonstrations improve clarification quality.

E. Experimental Pipeline

The complete pipeline is:

Database Selection → Query Preparation → Controlled Ambiguity Creation → Model Input → Ambiguity Detection → Clarification Generation → Simulated User Response → SQL Generation → SQL Execution → Evaluation

The database preparation stage will include tables, columns, relationships, and representative values. The same test set and database schemas will be used across systems to ensure a fair comparison.

VI. EXPERIMENTAL SETUP

A. Experimental Conditions

All systems will be evaluated on the same test set. For LLM-based systems, the model name/version, prompt, decoding parameters, maximum output length, and other relevant settings will be recorded.

The experiments will compare:

1. Rule-Based Clarification;
2. Direct LLM Text-to-SQL;
3. LLM + Clarification;
4. Few-Shot LLM, if implemented.

B. Evaluation Metrics

1) Ambiguity Classification

Accuracy, precision, recall, and F1-score will measure whether the system correctly identifies the ambiguity category.

2) Clarification Relevance

Human annotators will rate whether the generated question addresses the actual ambiguity using a 1–5 relevance scale.

3) Clarification Completeness

This metric measures whether the generated question requests all information necessary to resolve the ambiguity.

4) Human Preference

Human evaluators will compare outputs based on relevance, naturalness, helpfulness, and conciseness.

5) Text-to-SQL Performance

The final SQL will be evaluated using:

- **Exact Match (EM)**
- **Execution Accuracy (EX)**

The key comparison is:

Clarification Gain = SQL Accuracy after Clarification − Direct SQL Accuracy

C. Ablation Studies

At least one controlled ablation will be performed. The primary ablation will compare:

- Query only
- Query + database schema

Additional ablations will compare zero-shot versus few-shot prompting, prompts with versus without explicit ambiguity categories, and Bangla versus Banglish versus mixed-language queries.

D. Efficiency Analysis

Because clarification introduces an additional processing stage, efficiency will be measured using:

- Average response latency;
- Input token count;
- Output token count;
- Total token cost;
- Queries processed per minute; and
- Number of model calls.

The key efficiency question is:

Does the improvement in SQL accuracy justify the additional latency and token cost introduced by clarification?

E. Reproducibility

To ensure reproducibility, the project will record and, where permitted, release:

- Dataset and annotation guidelines;
- Train/dev/test splits;
- Prompts;
- Model names and versions;

- Decoding parameters;
- Database schemas;
- Evaluation scripts;
- Generated clarification questions;
- Generated SQL queries;
- Random seeds where applicable; and
- Hardware/software environment.

External datasets, models, software libraries, articles, and prompts will be properly cited in the final report.

VII. RESULTS AND ANALYSIS

Note: This section should contain actual experimental results after running the experiments. Do not report the expected results below as measured results.

A. Quantitative Results

The final report should use a table similar to the following:

Method	Ambiguity F1	Clarification Relevance	Completeness	SQL EM	SQL EX	Latency	Token Cost
Rule-Based	—	—	—	—	—	—	—
Direct LLM	—	—	—	—	—	—	—
LLM + Clarification	—	—	—	—	—	—	—
Few-Shot LLM	—	—	—	—	—	—	—

The primary analysis will determine whether clarification increases execution accuracy relative to direct SQL generation. Improvements should also be reported separately for each ambiguity type and language condition.

B. Ablation Analysis

The ablation results should determine whether schema information, few-shot examples, explicit ambiguity labels, and language choice significantly affect performance.

For example, if adding the schema improves ambiguity F1 and clarification relevance, this would indicate that database structure provides useful evidence for resolving ambiguity.

C. Language-Based Analysis

Performance should be compared across:

- Bangla;
- Banglish; and
- mixed Bangla-English.

Particular attention should be given to spelling variation and code-mixing in Banglish queries.

D. Qualitative Error Analysis

Representative incorrect outputs should be categorized as:

1. Wrong ambiguity detection;
2. Unnecessary clarification;
3. Incomplete clarification;
4. Irrelevant clarification;
5. Over-clarification;
6. Language misunderstanding;
7. Schema misunderstanding; and
8. Incorrect SQL after clarification.

For example, for the query *"সবচেয়ে ভালো customer দেখাও"*, asking *"আপনি কোন বছরের customer চান?"* is incorrect because the ambiguity concerns the definition of "ভালো customer," rather than the time period. A better clarification asks whether "good customer" means customers with the highest purchase amount, number of orders, or another criterion.

E. Expected Findings

The experimental hypothesis is that LLM-based clarification will outperform rule-based clarification, schema-aware systems will perform better than systems without schema information, and clarification will improve SQL execution accuracy compared with direct Text-to-SQL.

However, clarification is expected to introduce additional latency, model calls, and token usage. Banglish and vague-adjective/entity-identity ambiguity may also be more difficult than simple time-period ambiguity. These are hypotheses to be verified experimentally, rather than claims of completed results.

VIII. CONCLUSION

This project proposes **BanglaClarifySQL**, a clarification-aware framework for handling ambiguous Bangla and Banglish database queries. Instead of directly generating SQL from an incomplete or ambiguous natural-language question, the proposed system first identifies the ambiguity and asks an appropriate clarification question before generating the final SQL.

The study uses a self-constructed dataset, multiple meaningful baselines, a reproducible experimental pipeline, database and NLP evaluation metrics, controlled ablation studies, efficiency analysis, and qualitative error analysis. These components directly address the requirements of the proposed research project.

The central objective is to determine whether clarification can substantially improve SQL execution accuracy for Bangla and Banglish users while maintaining an acceptable computational cost. The expected contribution is a benchmark and taxonomy for ambiguity in Bangla/Banglish database queries, together with an empirical comparison of rule-based and LLM-based clarification approaches.

REFERENCES

[1] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman, Z. Zhang, and D. R. Radev, "Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task," in *Proc. 2018 Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2018.

[2] B. Wang, R. Shin, X. Liu, O. Polozov, and M. Richardson, "RAT-SQL: Relation-Aware Schema Encoding and Linking for Text-to-SQL Parsers," in *Proc. 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020.

[3] C. Scholak, N. Schucher, and T. Bahdanau, "PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models," in *Proc. 2021 Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2021.

[4] H. Pourreza and M. Rafiei, "DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction," in *Advances in Neural Information Processing Systems*, 2023.

[5] J. Li et al., "Can LLM Already Serve as a Database Interface? A BIg Bench for Large-Scale Database Grounded Text-to-SQLs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[6] Python Software Foundation, *Python Documentation*. [Software].

[7] Hugging Face, *Transformers: State-of-the-Art Machine Learning for PyTorch, TensorFlow, and JAX*. [Software].

[8] pandas development team, *pandas: Python Data Analysis Library*. [Software].

[9] SQLite, *SQLite Database Engine*. [Software].

[10] PostgreSQL Global Development Group, *PostgreSQL: The World's Most Advanced Open Source Relational Database*. [Software].